

네트워킹 Networking

Distributed Programming with JAVA (by Manning)
Core Java2 제2권 chap.3



네트워킹

- 네트워크 기초
- 소켓 프로그램의 기본 구조
- 서버 연결 – 클라이언트 작성
- 서버 구현
- 전자우편 보내기
- **URL** 연결
- 폼 데이터 포스팅
- 웹으로부터 정보 걷어들이기

네트워크 기본 개념

➤ 네트워크 기본 개념

- 프로토콜 (**protocol**) : 통신을 원하는 동등한 수준의 두 실체(**entity**) 간에 무엇을 어떻게 언제 통신할 것인가에 대해 서로 약속한 규정
- 통신망 구조(**network architecture**)
 - 통신 기능의 확장이나 새로운 통신 기술의 도입을 용이하도록 하기 위하여 프로토콜을 몇개의 계층으로 나눈 것.
 - 계층화된 구조를 사용함으로써 다른 계층에서의 구현에 상관없이 독립적인 구현이 가능하다.
 - 예) **OSI 7-layer, TCP/IP** 프로토콜 등
- **TCP/IP Suite**

응용(Application) 계층 : telnet, ftp, http, rlogin 등
전달 계층 : TCP & UDP
네트워크 계층 : IP
링크(Link) 계층:Ethernet, token ring 등)
물리(Physical) 계층

message

segment

datagram

frame

네트워크 기본 개념

- 응용 계층
 - 다양한 통신 서비스에 대한 인터페이스를 정의
 - **FTP, HTTP, Telnet, SMTP** 등
- 전달 계층
 - 응용 프로그램으로부터 전해진 데이터 메시지를 세그먼트 단위로 잘라 전송
 - 네트워크로부터 전해지는 데이터를 해당하는 응용프로그램으로 전달
 - **TCP(Transmission Control Protocol)**
 - » 순서제어와 에러 제어를 수행
 - » **SMTP, Telnet, HTTP, NFS** 등에서 사용
 - **UDP(User Datagram Protocol)**
 - » 순서제어와 에러 제어를 수행하지 않음.
 - » **internet telephony, multimedia data streaming** 등에 적당
- IP 계층
 - 목적 원격 컴퓨터까지의 적절한 경로를 통한 전송을 보장
 - 데이터그램 방식을 사용
- 링크 계층
 - 직접 연결된 두 호스트 사이의 데이터 전송을 수행

네트워크 기본 개념

➤ 소켓

- **IPC (Inter-Process Communication)**의 인터페이스를 파일의 입출력과 유사하게 함.
- 통신을 원하는 두개의 프로세스가 메시지 전송을 위한 소켓 스트림에 데이터를 읽고 씴으로써 데이터를 교환함.
- 소켓 주소
 - 인터넷 주소 (도메인 네임 또는 IP 주소)와 포트 번호로 구성
 - 0~65535의 포트 번호 중 0~1023은 슈퍼유저를 위하여 예약
- 종류
 - 데이터그램 통신 (**User Datagram Protocol**)
 - 비연결성 프로토콜 (목적 소켓 주소를 가진 패킷을 보냄)
 - 패킷의 길이는 64KB로 제한
 - 패킷의 분실 등을 관리하지 않는다.
 - 스트림 통신 (**Transfer Control Protocol**)
 - 두 소켓 사이의 연결이 이루어진 후에 통신이 이루어짐.
 - 클라이언트의 연결 요청을 서버 소켓이 수락 (**accept**) 함으로서 연결이 성립
 - 연결 후에는 주소 없이 양방향 데이터 전송이 가능
 - 신뢰성 있는 통신을 제공

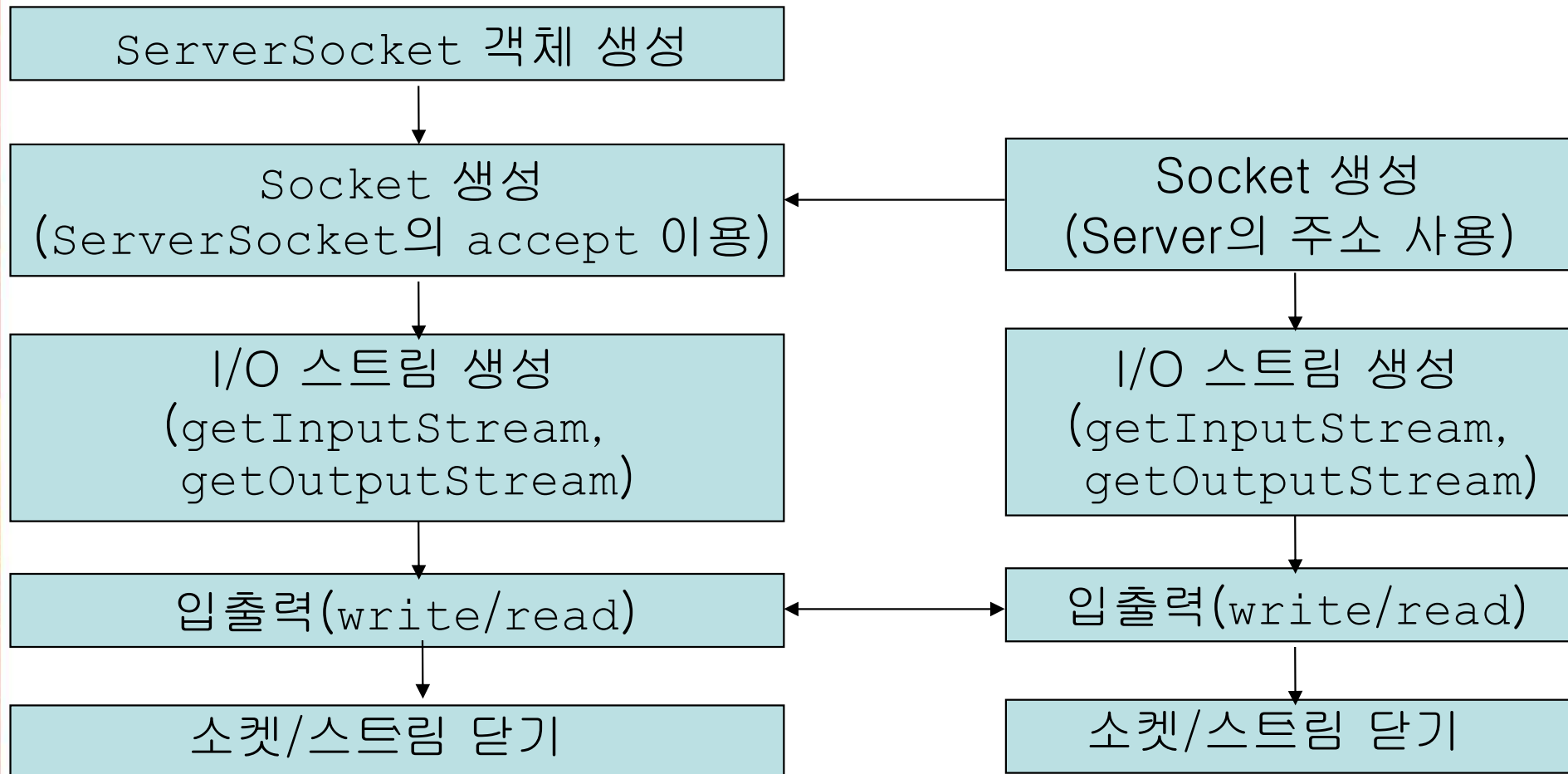
소켓 프로그램의 기본 구조 : TCP

➤ TCP 소켓 프로그래밍

- **Socket** 객체를 생성하여 사용
- 절차

(서버)

(클라이언트)



소켓 프로그램의 기본 구조 : TCP

■ 소켓 열기

```
Socket myClient = null;
try {
    myClient = new Socket("203.253.145.250", 5000);
} catch (UnknownHostException uhe) {
    uhe.printStackTrace();
} catch (IOException e) {
    e.printStackTrace();
}
```

```
ServerSocket myService = null;
try {
    myService = new ServerSocket(5000);
} catch (IOException e) {
    e.printStackTrace();
}
```

```
Socket serviceClient = null;
try {
    serviceClient = myService.accept();
} catch (IOException iex) {
    iex.printStackTrace();
}
```


소켓 프로그램의 기본 구조 : TCP

- 입출력 스트림의 생성

```
BufferedReader is = null;
try {
    is = new BufferedReader(new InputStreamReader(
                                myClient.getInputStream()));
} catch (IOException ioe) { ioe.printStackTrace();}

DataOutputStream os = null;
try {
    os = new DataOutputStream(myClient.getOutputStream());
} catch (IOException ioe) { ioe.printStackTrace(); }
```

```
BufferedReader is = null;
try {
    is = new BufferedReader(new InputStreamReader(
                                serviceClient.getInputStream()));
} catch (IOException ioe) { ioe.printStackTrace();}

DataOutputStream os = null;
try {
    os = new DataOutputStream(serviceClient.getOutputStream());
} catch (IOException ioe) { ioe.printStackTrace(); }
```


소켓 프로그램의 기본 구조 : TCP

■ 입/출력을 사용한 통신

```
...  
os.writeBytes("Hello.\n");  
String reply = is.ReadLine();  
System.out.print(reply);  
...
```

```
...  
String reply = is.ReadLine();  
System.out.print(reply);  
os.writeBytes("Hello2\n");  
...
```

■ 소켓 닫기

```
try {  
    os.close();  
    is.close();  
    myClient.close();  
} catch (IOException io) {  
    io.printStackTrace();  
}
```

```
try {  
    os.close();  
    is.close();  
    serviceClient.close();  
} catch (IOException io) {  
    io.printStackTrace();  
}
```

소켓 프로그램의 기본 구조 : UDP

➤ UDP 소켓 프로그래밍

▪ 특징

- 패킷 단위의 통신을 함.
- 각각의 목적지 주소와 데이터로 이루어진 패킷을 생성하여 소켓으로 전송함. (소켓에 목적지를 고정할 수도 있음)

▪ DatagramSocket의 생성

```
//host1.hangkong.ac.kr에서 동작하는 프로그램  
DatagramSocket ds = new DatagramSocket(5555);
```

```
//host2.hangkong.ac.kr에서 동작하는 프로그램  
DatagramSocket ds = new DatagramSocket(6666);
```

- 포트 번호를 지정하지 않을 수도 있다. (보내기만 할 경우)

```
DatagramSocket ds = new DatagramSocket();  
int portNumber = ds.getLocalPort();
```

소켓 프로그램의 기본 구조 : UDP

- 패킷의 생성과 송수신

```
byte buf[] = {'h','e','l','l','o'};
InetAddress address =
    InetAddress.getByName("host2.hangkong.ac.kr");
DatagramPacket packet =
    new DatagramPacket(buf,buf.length,address,6666);
ds.send(packet);

byte buf2[] = new byte[256];
DatagramPacket packet=new DatagramPacket(buf2,buf2.length);
ds.receive(packet);
```

```
byte buf[] = new byte[256];
DatagramPacket packet=new DatagramPacket(buf,buf.length);
ds.receive(packet);

byte buf2[] = {'h','i'};
InetAddress address =
    InetAddress.getByName("host1.hangkong.ac.kr");
DatagramPacket packet =
    new DatagramPacket(buf2,buf2.length,address,5555);
ds.send(packet);
```

소켓 프로그램의 기본 구조 : UDP

➤ UDP 프로그램 예제

```
import java.io.*;
import java.net.*;

public class DgramReply {
    public final static int MY_PORT = 5001;
    public static void main(String argv[]) throws Exception {
        DatagramSocket ds = null;
        byte buf[] = new byte[256];
        ds = new DatagramSocket(MY_PORT);
        DatagramPacket packet=new DatagramPacket(buf, buf.length);

        ds.receive(packet);
        System.out.println("Got from client: ");
        for (int i=0; i<packet.getLength(); i++)
            System.out.print((char)buf[i]);
        ds.send(packet);

        ds.close();
    }
}
```

소켓 프로그램의 기본 구조 : UDP

```
import java.io.*;
import java.net.*;

public class DgramSender {
    public final static int MY_PORT = 5000;
    public final static int TO_PORT = 5001;
    public static void main(String argv[]) throws Exception {
        DatagramSocket ds = new DatagramSocket(MY_PORT);
        byte[] buf = new byte[256];
        System.out.print("Please input a keyword: ");
        int n = System.in.read(buf);
        InetAddress addr = InetAddress.getByName("jsahn");
        DatagramPacket packet =
            new DatagramPacket(buf, n, addr, TO_PORT);
        ds.send(packet);
        DatagramPacket packet2 = new DatagramPacket(buf, buf.length);
        ds.receive(packet2);
        buf = packet2.getData();
        for (int i=0; i<packet2.getLength(); i++)
            System.out.print((char)buf[i]);
        ds.close();
    }
}
```

서버에 연결하기

```
import java.io.*;
import java.net.*;

public class SocketTest
{   public static void main(String[] args)
    {   try
        {   Socket s = new Socket("time-A.timefreq.bldrdoc.gov",
                                13);

            BufferedReader in = new BufferedReader
                (new InputStreamReader(s.getInputStream()));
            boolean more = true;
            while (more)
            {   String line = in.readLine();
                if (line == null) more = false;
                else
                    System.out.println(line);
            }
        }
        catch (IOException e)
        {   System.out.println("Error" + e); }
    }
}
```


서버에 연결하기 : 무한 블록 해결(timeout 설정)

- 클라이언트가 블록 되는 경우
 - **Socket** 객체 생성시 서버에 연결이 되지 않는 경우
 - **read**의 수행 시 데이터가 전달되지 않을 경우
- **read**로 인한 블록의 해결
 - 해당 소켓 객체에 **setSoTimeout** 메소드로 타임 아웃 값을 설정한다.

```
socket a = new Socket(. . .);  
s.setSoTimeout(10000); //  
...  
try {  
    while((String line = in.readLine()) != null) {  
        { // process line }  
    }  
catch (SocketException exception) {  
    // time out occurred !  
}  
...
```


서버의 구현 : 순차적인 서비스

➤ Echo Server

```
import java.io.*;
import java.net.*;
public class EchoServer
{   public static void main(String[] args )
    {   try
        {   ServerSocket s = new ServerSocket(8189);
            Socket incoming = s.accept( );
            BufferedReader in = new BufferedReader
                (new InputStreamReader(incoming.getInputStream()));
            PrintWriter out = new PrintWriter
                (incoming.getOutputStream(), true /* autoFlush */);
            out.println( "Hello! Enter BYE to exit." );
            boolean done = false;
            while (!done)
            {   String line = in.readLine();
                if (line == null) done = true;
                else
                {   out.println("Echo: " + line);
                    if (line.trim().equals("BYE")) done = true;
                }
            }
            incoming.close();
        }catch (Exception e) { System.out.println(e); }
    }
}
```

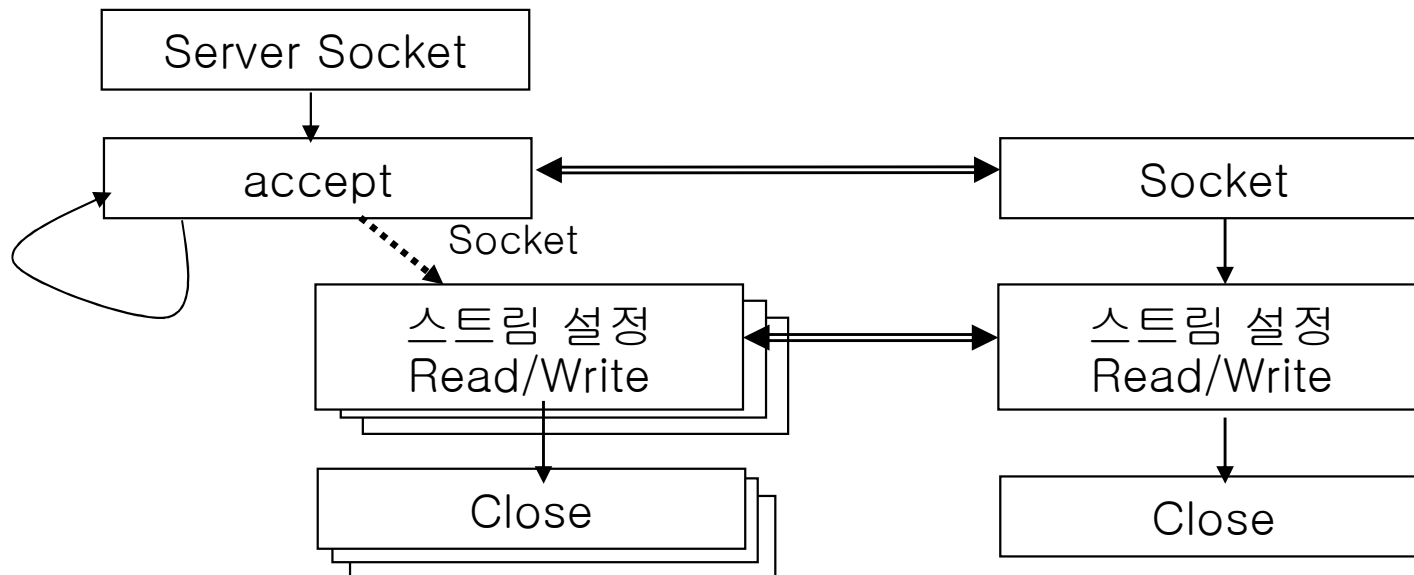
서버의 구현 : 병렬 쓰레딩의 이용

➤ 복수 클라이언트의 지원

- 동시에 여러 개의 클라이언트가 연결되기를 원할 경우 병렬 쓰레드를 사용하여 여러 개의 클라이언트를 동시에 지원 가능

```
while (true) {  
    Socket incoming = s.accept();  
    Thread t = new ThreadedEchoHandler(incoming);  
    t.start();  
}
```

- 하나의 **ServerSocket**을 사용하여 여러 개의 연결된 **Socket**을 생성함.
- 각각의 쓰레드의 **run** 메소드에 연결된 후의 서버의 작업을 기술



서버의 구현 : 병렬 쓰레딩의 이용

▪ ThreadedEchoServer.java

```
import java.io.*;
import java.net.*;

public class ThreadedEchoServer
{   public static void main(String[] args )
    {   int i = 1;
        try
        {   ServerSocket s = new ServerSocket(8189);

            for (;;)
            {   Socket incoming = s.accept( );
                System.out.println("Spawning " + i);
                new ThreadedEchoHandler(incoming, i).start();
                i++;
            }
        }
        catch (Exception e)
        {   System.out.println(e);
        }
    }
}
```

서버의 구현 : 병렬 쓰레딩의 이용

```
class ThreadedEchoHandler extends Thread
{   public ThreadedEchoHandler(Socket i, int c)
    {   incoming = i; counter = c; }
    public void run()
    {   try
        {   BufferedReader in = new BufferedReader
            (new InputStreamReader(incoming.getInputStream()));
            PrintWriter out = new PrintWriter
            (incoming.getOutputStream(), true);
            out.println("Hello! Enter BYE to exit.");
            boolean done = false;
            while (!done)
            {   String str = in.readLine();
                if (str == null) done = true;
                else {
                    out.println("Echo (" + counter + "): " + str);
                    if (str.trim().equals("BYE")) done = true;
                }
            }
            incoming.close();
        }
        catch (Exception e) { System.out.println(e); }
    }
    private Socket incoming;
    private int counter;
}
```